

SQL Server Guide: Data Languages, Operations and Joins

This guide covers SQL Server data languages (DQL, DML, DCL), common operations with examples, and SQL joins. All examples use SQL Server syntax and include USE database statements where appropriate.

1. SQL Data Languages

DQL (Data Query Language)

Used to retrieve data from database tables.

```
USE EmployeeDB; SELECT * FROM Employees;
```

DML (Data Manipulation Language)

Used to insert, update and delete data.

```
USE EmployeeDB; INSERT INTO Employees(EmployeeName, Salary) VALUES ('John', 50000);  
UPDATE Employees SET Salary = 55000 WHERE EmployeeName = 'John'; DELETE FROM  
Employees WHERE EmployeeName = 'John';
```

DCL (Data Control Language)

Used to control user permissions.

```
USE EmployeeDB; GRANT SELECT ON Employees TO User1; REVOKE SELECT ON Employees  
FROM User1;
```

2. SQL Operations

Create Database

Creates a new database.

```
CREATE DATABASE EmployeeDB;
```

Use Database

Selects the database for execution.

```
USE EmployeeDB;
```

Create Table

Creates a new table.

```
USE EmployeeDB; CREATE TABLE Employees( EmployeeId INT PRIMARY KEY, EmployeeName  
VARCHAR(100), Salary DECIMAL(10,2) );
```

Insert

Adds records into a table.

```
USE EmployeeDB; INSERT INTO Employees(EmployeeId, EmployeeName, Salary) VALUES (1,  
'John', 50000);
```

Select

Retrieves data from a table.

```
USE EmployeeDB; SELECT * FROM Employees;
```

Where

Filters records.

```
USE EmployeeDB; SELECT * FROM Employees WHERE Salary > 40000;
```

Order By

Sorts records.

```
USE EmployeeDB; SELECT * FROM Employees ORDER BY Salary DESC;
```

Update

Modifies existing records.

```
USE EmployeeDB; UPDATE Employees SET Salary = 60000 WHERE EmployeeId = 1;
```

Delete

Removes records.

```
USE EmployeeDB; DELETE FROM Employees WHERE EmployeeId = 1;
```

Truncate

Removes all rows but keeps structure.

```
USE EmployeeDB; TRUNCATE TABLE Employees;
```

Drop Table

Deletes table structure and data.

```
USE EmployeeDB; DROP TABLE Employees;
```

Alter Table

Modifies table structure.

```
USE EmployeeDB; ALTER TABLE Employees ADD Email VARCHAR(100);
```

3. SQL Joins

Assume the following tables: Employees(EmployeeId, EmployeeName, DepartmentId)
Departments(DepartmentId, DepartmentName)

INNER JOIN

Returns matching records from both tables.

```
USE EmployeeDB; SELECT e.EmployeeName, d.DepartmentName FROM Employees e INNER  
JOIN Departments d ON e.DepartmentId = d.DepartmentId;
```

LEFT JOIN

Returns all records from left table and matching records from right table.

```
USE EmployeeDB; SELECT e.EmployeeName, d.DepartmentName FROM Employees e LEFT  
JOIN Departments d ON e.DepartmentId = d.DepartmentId;
```

RIGHT JOIN

Returns all records from right table and matching records from left table.

```
USE EmployeeDB; SELECT e.EmployeeName, d.DepartmentName FROM Employees e RIGHT  
JOIN Departments d ON e.DepartmentId = d.DepartmentId;
```

FULL OUTER JOIN

Returns all matching and non-matching records.

```
USE EmployeeDB; SELECT e.EmployeeName, d.DepartmentName FROM Employees e FULL  
OUTER JOIN Departments d ON e.DepartmentId = d.DepartmentId;
```

SELF JOIN

Joins a table with itself.

```
USE EmployeeDB; SELECT e1.EmployeeName, e2.EmployeeName AS Manager FROM  
Employees e1 JOIN Employees e2 ON e1.EmployeeId = e2.EmployeeId;
```

CROSS JOIN

Returns Cartesian product of both tables.

```
USE EmployeeDB; SELECT e.EmployeeName, d.DepartmentName FROM Employees e CROSS  
JOIN Departments d;
```